Automated DevSecOps CI/CD Pipeline



DISCLAIMER

This video/docx is created strictly for educational, ethical, and informational purposes only. Nothing shown or discussed in this content is intended to promote, assist, or enable hacking, illegal activities, unauthorized access, evasion of law enforcement, or any type of cybercrime. All demonstrations are performed in a controlled environment, using safe, non-actionable examples that do not provide step-by-step instructions for misuse.

The goal of this video/docx is to help viewers understand cybersecurity concepts, improve digital safety, recognize threats, and learn how modern privacy technologies work in a legal and responsible way.

Any tools, terms, or techniques mentioned should be used only for defensive research, academic learning, penetration testing on systems you own or are authorized to test, or for privacy protection within the boundaries of the law.

Attempting to use these techniques for illegal or unethical purposes may result in criminal charges, account bans, data loss, civil penalties, or permanent damage to networks and devices.

Neither the creator of this video/docx nor any associated platforms are responsible for misuse, misinterpretation, or illegal actions taken by viewers.

By watching this video/docx, you agree to use the information responsibly, ethically, and in compliance with all applicable laws.

If you are interested in cybersecurity, always follow ethical hacking guidelines, obtain proper permissions, and respect privacy and security standards at all times.

Enterprise DevSecOps & Cloud Security Baseline: Automated IaC Hardening & CI/CD Governance (AWS & Terraform)

1. Project Overview & Business Problem

- **Domain:** Cloud Infrastructure Security, Infrastructure as Code (IaC), DevSecOps CI/CD Governance
- **Target Platforms & Tools:** AWS (S3, KMS, IAM), HashiCorp Terraform, GitHub Actions, Aqua Security Trivy, TruffleHog
- **Compliance Standards:** SOC 2 Type II (CC6.1, CC6.6, CC6.7, CC7.1), ISO/IEC 27001:2022 (A.8.13, A.8.24, A.8.28), CIS AWS Benchmark 2.1
- **Problem Statement:** In fast-moving cloud environments, infrastructure misconfigurations (unencrypted storage, public read/write exposure, missing TLS enforcement) and hardcoded credentials represent the leading causes of cloud breaches. Manually auditing code before deployment fails to scale and creates human error bottlenecks.
- **Core Solution:** Engineered a shift-left DevSecOps automation pipeline that gates every code change using static analysis, automated secret detection, and strict policy-as-code rules. Cloud storage assets are provisioned using hardened Terraform blueprints enforcing Customer-Managed KMS encryption, mandatory TLS 1.2+ transport, and complete public exposure isolation.

2. Technical Architecture & Component Breakdown

A. Infrastructure as Code (Terraform)

- KMS Key Management (aws_kms_key): Dedicated Customer-Managed Key (CMK) configured with a 30-day soft deletion window and mandatory automated 365-day key rotation to fulfill SOC 2 / ISO 27001 cryptographic governance.
- **Hardened S3 Storage (`aws_s3_bucket`):** Immutable audit storage container tagged with enterprise data classification labels (`Restricted`, `SOC2-ISO27001`).
- **Multi-Layer Public Access Shield (`aws_s3_bucket_public_access_block`):** Complete blockage of public ACLs, bucket policies, and cross-account access points aligning with CIS Benchmark 2.1.5.
- **Server-Side Encryption Enforcement (`aws_s3_bucket_server_side_encryption_configuration`):** Mandated `aws:kms` algorithm linked to the CMK with `bucket_key_enabled = true` to lower AWS KMS request overhead by up to 99% while maintaining cryptographic isolation.
- **Object Versioning & Anti-Tamper (`aws_s3_bucket_versioning`):** Active versioning enabled to prevent ransomware alteration, accidental destruction, and unauthorized overwrites.
- **Transport Encryption Policy (`aws_s3_bucket_policy`):** Strict explicit Deny policy applied to any non-HTTPS traffic where aws:SecureTransport = false, terminating plaintext requests at the edge.

B. CI/CD DevSecOps Pipeline (GitHub Actions)

The workflow (.github/workflows/security.yml) executes three sequential gating jobs on every commit and pull request to main:

graph LR

A[Git Push / PR] --> B[Job 1: TruffleHog Secret Scan]

B -->|Passed / No Exposed Keys| C[Job 2: Trivy IaC Misconfiguration Scan]

C -->|Passed / No High or Critical CVEs| D[Job 3: Terraform Lint & Validate]

D -->|Passed / Canonical Form| E[Green Status Gate / Production Ready]

B -->|Secret Detected| F[FAIL: Block Pipeline]

C -->|Misconfiguration Found| F

D -->|Syntax Error| F

1. Job 1 — Secrets Detection (TruffleHog): Scans the repository tree for leaked private credentials, verified API tokens, and private keys to eliminate secrets-in-code vulnerabilities.

2. Job 2 — Static Infrastructure Vulnerability Scan (Aqua Security Trivy): Scans the Terraform codebase against CIS benchmarks and cloud vulnerability databases. Configured with --severity HIGH,CRITICAL and exit-code: 1 to halt the build automatically if any misconfiguration is identified.

3. Job 3 — Terraform Validation & Linting: Executes terraform fmt -check to enforce canonical formatting standards and terraform validate to verify provider configuration integrity before merge approval.

3. Engineering Challenges & Resolving Pipeline Failures

Documenting actual troubleshooting and resolution demonstrates authentic DevSecOps engineering depth:

Challenge 1: GitHub PAT Workflow Permission Scope

Symptom: Git refused push with ! [remote rejected] main -> main (refusing to allow a Personal Access Token to create or update workflow '.github/workflows/security.yml' without 'workflow' scope).

Root Cause: GitHub requires granular access token delegation for actions touching .github/workflows/* to prevent privilege escalation via pipeline manipulation.

Remediation: Regenerated a GitHub Classic PAT with explicit workflow and repo scopes enabled, ensuring administrative pipeline management complied with least-privilege token access.

Challenge 2: TruffleHog Git Baseline Diff Collision

Symptom: Pipeline failed at Job 1 with Error: BASE and HEAD commits are the same. TruffleHog won't scan anything. Process completed with exit code 1.

Root Cause: TruffleHog's default git-diff comparator requires two distinct commits to calculate changes. On an initial commit push, base and head point to the identical commit hash.

Remediation: Refactored the GitHub Actions step parameter to use path: ./ with --only-verified extra arguments, transitioning the scanner to inspect the codebase directly without relying on commit history diffs.

Challenge 3: Trivy IaC Compliance Finding (AVD-AWS-0132)

Symptom: Job 2 tripped with exit code 1: Encryption using KMS keys provides protection for your S3 buckets. To gain greater control over encryption, such as key rotation, use SSE-KMS. See
https://avd.aquasec.com/misconfig/aws-0132.

Root Cause: Trivy enforces strict SOC 2 and ISO 27001 key governance rules. Standard AWS-managed S3 encryption (AES256 or default aws/s3) prevents the organization from managing key lifecycles, auditing access policies, or configuring annual rotation.

Remediation: Upgraded the Terraform architecture by provisioning an independent aws_kms_key resource with enable_key_rotation = true, linking its ARN directly into apply_server_side_encryption_by_default.kms_master_key_id. Re-running the pipeline confirmed zero high/critical vulnerabilities.

4. Compliance & Controls Mapping Matrix

Security Domain	Technical Implementation	Governance Framework Reference
Cryptographic Protection	Customer-Managed KMS Key (`aws_kms_key`) with automated rotation and SSE-KMS bucket policy	**ISO/IEC 27001:2022 A.8.24** (Use of Cryptography) **SOC 2 CC6.1** (Logical Access Controls) **SOC 2 CC6.6** (Boundary Defense)
Boundary Protection	Strict public access block (aws_s3_bucket_public_access_block) eliminating public read/write ACLs and policies	CIS AWS Benchmark 2.1.5
Data in Transit	Explicit bucket policy denial for requests where `aws:SecureTransport = false`	**SOC 2 CC6.7** (Transmission Protection) CIS AWS Benchmark 2.1.2
Data Integrity & Continuity	Object versioning enabled (aws_s3_bucket_versioning) to prevent accidental deletion and ransomware tampering	**ISO/IEC 27001:2022 A.8.13** (Information Backup) **SOC 2 CC7.1** (System Operations)
Shift-Left Security Verification	Pre-merge automated scanning with Aqua Security Trivy for high/critical IaC misconfigurations	ISO/IEC 27001:2022 A.8.28 (Secure Coding / Shift-Left) DevSecOps Best Practices
Credential Hygiene	Automated TruffleHog scanning on every pull request to intercept exposed API keys and secrets	OWASP Top 10 A07:2021 (Identification & Auth Failures) SOC 2 CC6.1

2. Architecture & Pipeline Sequence

A. Infrastructure Topology

```
graph TD

    User([Client / API Request]) -->|Enforce TLS 1.2+ HTTPS| BucketPolicy[AWS S3 Bucket Policy]

    subgraph "AWS Cloud Infrastructure"

        subgraph "Amazon S3 Boundary"

            BucketPolicy --> PAB[S3 Public Access Block]

            PAB --> CoreBucket[Hardened S3 Storage Bucket]

            subgraph "Security Controls"

                CoreBucket --- SSE[SSE-KMS Encryption]

                CoreBucket --- Versioning[Object Versioning Enabled]

                CoreBucket --- Tags[SOC 2 / ISO 27001 Tags]

            end

        end

        subgraph "AWS Key Management Service (KMS)"

            KMSKey[Customer-Managed Key CMK] -.->|Auto-Rotation 365 Days| SSE

        end

    end

    UnauthorizedPublicUser([Internet Public / Plaintext HTTP]) -.->|DENY 403| BucketPolicy
```

B. CI/CD DevSecOps Flow

```
sequenceDiagram

    autonumber
```

```
actor Engineer as Cloud/SecOps Engineer
participant GitHub as GitHub VCS
participant Runner as GitHub Actions Runner
participant TruffleHog as TruffleHog (Secrets)
participant Trivy as Trivy (IaC Scanner)
participant Terraform as Terraform CLI

Engineer->>GitHub: Push code to main / Open PR
GitHub->>Runner: Trigger DevSecOps Security Pipeline
Runner->>TruffleHog: Scan commit tree for exposed keys/tokens
alt Secret Detected
    TruffleHog-->>Runner: Hard Fail (Exit Code 1)
    Runner-->>Engineer: Block PR / Leaked Secret Alert
else Clean Commit
    TruffleHog->>Runner: Pass
    Runner->>Trivy: Scan Terraform against CIS, SOC 2 & ISO benchmarks
    alt High / Critical Misconfiguration Found
        Trivy-->>Runner: Hard Fail (Exit Code 1)
        Runner-->>Engineer: Reject Infrastructure Change
    else Compliance Verified
        Trivy->>Runner: Pass
        Runner->>Terraform: Validate syntax & canonical formatting
        Terraform-->>Runner: Pass validation
        Runner-->>GitHub: Mark Build Checks Green (✔)
```

```
    end

  End
```

3. Full Implementation Source Code

A. Terraform Infrastructure as Code (`terraform/main.tf`)

```
terraform {
  required_version = ">= 1.5.0"
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = var.aws_region
}

# KMS Key with automated 365-day rotation for SOC 2 (CC6.1) and ISO 27001 (A.8.24)
resource "aws_kms_key" "s3_key" {
  description             = "Customer-managed KMS key for S3 bucket encryption"
  deletion_window_in_days = 30
  enable_key_rotation     = true

  tags = {
    Environment     = var.environment
    ComplianceScope = "SOC2-ISO27001"
    ManagedBy       = "Terraform"
  }
}

# Core S3 Storage Bucket
resource "aws_s3_bucket" "secure_storage" {
  bucket_prefix = "corp-audit-logs-"
  force_destroy = false

  tags = {
    Environment       = var.environment
    ComplianceScope    = "SOC2-ISO27001"
```

```
    DataClassification = "Restricted"
    ManagedBy          = "Terraform"
  }
}

# S3 Public Access Block - CIS AWS Benchmark 2.1.5 & SOC 2 CC6.6
resource "aws_s3_bucket_public_access_block" "public_block" {
  bucket = aws_s3_bucket.secure_storage.id

  block_public_acls       = true
  block_public_policy     = true
  ignore_public_acls      = true
  restrict_public_buckets = true
}

# S3 Versioning - ISO 27001 A.8.13 & SOC 2 CC7.1
resource "aws_s3_bucket_versioning" "versioning" {
  bucket = aws_s3_bucket.secure_storage.id

  versioning_configuration {
    status = "Enabled"
  }
}

# S3 Server-Side Encryption using Customer-Managed KMS Key
resource "aws_s3_bucket_server_side_encryption_configuration" "secure_storage_encryption"
{
  bucket = aws_s3_bucket.secure_storage.id

  rule {
    apply_server_side_encryption_by_default {
      kms_master_key_id = aws_kms_key.s3_key.arn
      sse_algorithm     = "aws:kms"
    }
    bucket_key_enabled = true
  }
}

# S3 TLS 1.2+ Transport Enforcement Policy - CIS AWS Benchmark 2.1.2 & SOC 2 CC6.7
resource "aws_s3_bucket_policy" "enforce_tls" {
  bucket = aws_s3_bucket.secure_storage.id

  policy = jsonencode({
    Version = "2012-10-17"
```

```
    Statement = [
     {
       Sid       = "EnforceTLSRequestsOnly"
       Effect    = "Deny"
       Principal = "*"
       Action    = "s3:*"
       Resource = [
         aws_s3_bucket.secure_storage.arn,
         "${aws_s3_bucket.secure_storage.arn}/*"
       ]
       Condition = {
         Bool = {
           "aws:SecureTransport" = "false"
         }
       }
     }
   ]
  })
}
```

B. Input Variables (`terraform/variables.tf`)

```
variable "aws_region" {
  type        = string
  description = "Target AWS deployment region"
  default     = "eu-north-1"
}

variable "environment" {
  type        = string
  description = "Target deployment lifecycle tier"
  default     = "production"
}
```

C. DevSecOps CI/CD Pipeline Workflow (`.github/workflows/security.yml`)

```
name: "DevSecOps Security Pipeline"

on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

permissions:
  contents: read
```

```
  security-events: write

jobs:
  # Stage 1: Automated Secrets Detection
  secret-scan:
    name: "Secrets Detection (TruffleHog)"
    runs-on: ubuntu-latest
    steps:
      - name: Checkout Code
        uses: actions/checkout@v4
        with:
          fetch-depth: 0

      - name: Scan Repository for Leaked Secrets
        uses: trufflesecurity/trufflehog@main
        with:
          path: ./
          extra_args: --only-verified

  # Stage 2: Static IaC Security & Misconfiguration Analysis
  iac-security:
    name: "IaC Misconfiguration Scan (Trivy)"
    runs-on: ubuntu-latest
    needs: secret-scan
    steps:
      - name: Checkout Code
        uses: actions/checkout@v4

      - name: Run Trivy IaC Scanner
        uses: aquasecurity/trivy-action@master
        with:
          scan-type: "config"
          scan-ref: "./terraform"
          severity: "HIGH,CRITICAL"
          exit-code: "1"

  # Stage 3: Terraform Linting and Syntax Validation
  terraform-lint:
    name: "Terraform Validation & Linting"
    runs-on: ubuntu-latest
    needs: iac-security
    steps:
      - name: Checkout Code
        uses: actions/checkout@v4
```

```
- name: Setup HashiCorp Terraform
  uses: hashicorp/setup-terraform@v3
  with:
    terraform_version: 1.6.3

- name: Check Canonical Formatting
  run: terraform -chdir=terraform fmt -check

- name: Validate Configuration Syntax
  run: |
    terraform -chdir=terraform init -backend=false
    terraform -chdir=terraform validate
```

Engineering Challenges & Resolving Pipeline Failures

Issue 1: GitHub PAT Workflow Scope Block

- **Log Error:** ! [remote rejected] main -> main (refusing to allow a Personal Access Token to create or update workflow '.github/workflows/security.yml' without 'workflow' scope)
- **Root Cause:** GitHub requires granular access token delegation for workflows touching .github/workflows/ to prevent unintended pipeline manipulation.
- **Resolution:** Re-scoped the GitHub Personal Access Token to explicitly grant the workflow capability alongside repo access.

Issue 2: TruffleHog Invalid Parameter Argument

- **Log Error:** trufflehog: error: unknown long flag '--directory', try --help

Encryption using KMS keys provides protection for your S3 buckets.

To gain greater control over encryption, such as key rotation, use SSE-KMS.

See https://avd.aquasec.com/misconfig/aws-0132

main.tf:44-53

```[cite: 20]

**Root Cause:** Default AWS-managed encryption (AES256 / aws/s3) does not provide granular access policies or mandatory key rotation required by SOC 2 CC6.1 and ISO 27001 A.8.24.

**Resolution:** Defined a dedicated `aws_kms_key` resource with `enable_key_rotation = true` and configured `kms_master_key_id = aws_kms_key.s3_key.arn` with `bucket_key_enabled = true`. Subsequent pipeline runs passed with zero high or critical findings
```

Bullet Points

- **Automated DevSecOps Pipeline (AWS & Terraform):** Architected a continuous integration security gate using GitHub Actions, integrating TruffleHog secret scanning and Aqua Security Trivy to enforce zero high/critical IaC vulnerabilities prior to infrastructure merge.